Reti e Laboratorio: Modulo Laboratorio III
CONNECTIONS: Collega le parole e vinci!
Progetto di Fine Corso A.A. 2025/26- Corsi A - B

Versione 1.0 - 11 Dicembre 2025

1. Connections: descrizione del gioco

Connections (https://en.wikipedia.org/wiki/The_New_York_Times_Connections) è un gioco la cui finalità è trovare aggregazioni di parole in base a qualche loro caratteristica. Il gioco è stato sviluppato e pubblicato nel giugno 2023 dal The New York Times come parte di The New York Times Games.

All'inizio di ogni partita, viene generato un insieme di 16 parole, suddivise in 4 gruppi tematici distinti composti da 4 parole ciascuno. Le parole generate vengono quindi inviate in ordine casuale a tutti i giocatori. Ogni giocatore può inviare una o più proposte di raggruppamento, consistenti in quadruple di parole dall'elenco ricevuto aggregate rispetto a qualche caratteristica che le accomuni. Tali proposte vengono valutate dal gestore del gioco, che determina se ogni quadrupla corrisponde a uno dei gruppi previsti. Ogni giocatore ha un numero massimo di 4 proposte sbagliate per ogni partita. Le partite sono suddivise per periodi temporali. Durante un certo periodo una ed una sola partita è attiva per tutti i giocatori che vogliono giocare. Un giocatore, anche se ha già vinto o perso, deve aspettare lo scadere del tempo della partita prima di poter accedere alla partita successiva. Un giocatore vince la partita se invia tre gruppi corretti di 4 parole (poiché l'ultimo gruppo è implicito per esclusione) prima dello scadere del tempo e prima di raggiungere il limite di 4 proposte sbagliate. Per ogni partita a cui partecipa un giocatore ottiene il seguente punteggio:

- 0 per partecipare a partita senza inviare nessuna proposta;
- il giocatore ottiene un bonus dipendente dal numero di proposte corrette:
    - +6 se indovina esattamente una proposta corretta;
    - +12 se indovina esattamente due proposte corrette;
    - +18 se indovina tre proposte corrette (e quindi vince la partita poiché l'ultima proposta è implicita);
- il giocatore ottiene una penalità di -4 per ogni proposta sbagliata inviata (quindi fino ad un massimo di -16 per una partita persa).

Se ad esempio un giocatore invia una proposta sbagliata, poi una proposta corretta, poi un'altra proposta sbagliata e poi il tempo scade e la partita finisce il punteggio per quel giocatore per quella partita sarà -2. Se invece un giocatore vince la partita (inviando 3 proposte corrette) ma fa anche 3 errori (inviando 3 proposte sbagliate) otterrà un punteggio di +6. In Fig.1-3 è mostrata una partita in diversi stadi di gioco.

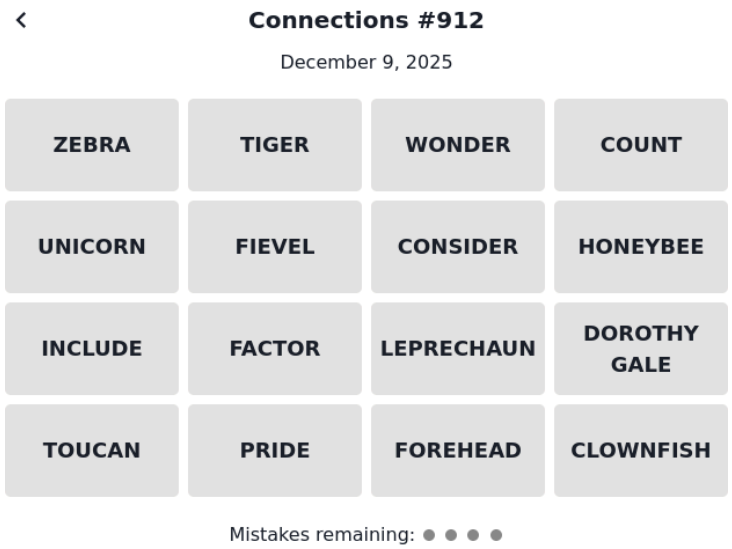


Fig. 1 Connections: uno scenario di gioco in cui il giocatore si è appena unito ad una partita. Al momento il giocatore ha accumulato 0 punti

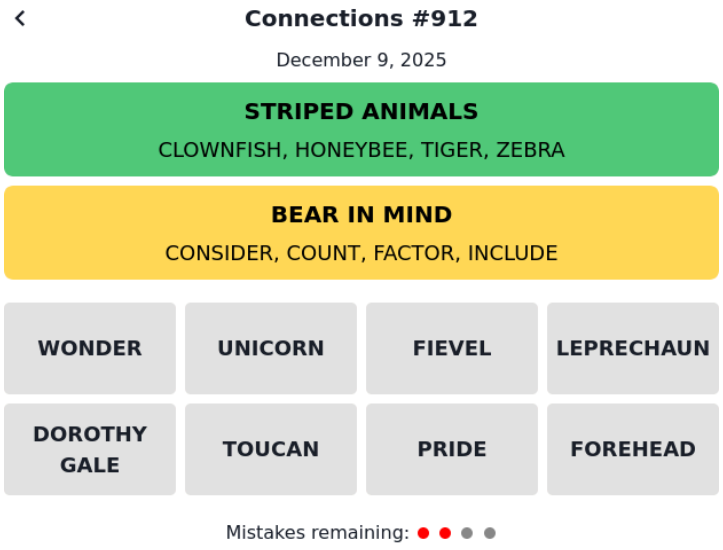


Fig. 2 Connections: uno scenario di gioco in cui il giocatore ha inviato due proposte corrette e due sbagliate (indicate dai punti rossi in basso).
Al momento il giocatore ha accumulato 4 punti



Fig. 3 Connections: uno scenario di gioco in cui il giocatore ha vinto la partita inviando tre proposte corrette e tre sbagliate (indicate dai punti rossi in basso). Con questa partita il giocatore ha accumulato 6 punti

2. Connections: l'architettura client server

Il sistema è strutturato secondo un modello *client–server* centralizzato, in cui il server coordina lo svolgimento della partita (poiché esiste un'unica partita globale a cui partecipano tutti i giocatori in ogni dato momento) e gestisce lo stato condiviso, mentre i client svolgono attività di input dai giocatori o di visualizzazione dello stato corrente e dei risultati.

2.1 Il Client

Il client rappresenta l'interfaccia tra il giocatore e il server. La sua funzione principale è permettere all'utente di partecipare al gioco, ricevere informazioni, inviare proposte di gruppi di parole e ricevere feedback per ogni azione.

Il client deve offrire le seguenti operazioni.

- Operazioni di gestione dell'utente giocatore:
  - *registrazione*: un utente può registrarsi indicando un nome utente e password sse il nome utente non è già registrato;
  - *aggiornamento credenziali:* un utente può aggiornare il proprio nome utente o la password collegata ad un nome utente se è in grado di dimostrare di conoscere la password attualmente associata a quel nome utente;

    - *login*: un utente può effettuare il login se conosce la password corretta relativa ad un certo nome utente. Effettuare il login automaticamente fa accedere alla partita attualmente in corso ricevendone le informazioni necessarie (set di parole, proposte corrette eventualmente già inviate da quell'utente, numero di errori, tempo rimanente, punteggio corrente);
    - *logout*: effettuare il logout informa interrompe la possibilità di ricevere notifiche dal server ed inviare proposte sulla partita corrente al server.
- Operazioni di gestione della partita corrente (solo se logged in):
    - *inviare proposta*: invia una nuova proposta di 4 parole tra quelle non ancora correttamente inserite in un gruppo da quell'utente. All'invio segue risposta del server per notificare la correttezza o meno della proposta;
    - *richiedere stato/esito partita*: l'utente specifica l'id univoco della partita a cui è interessato oppure che è interessato alla partita in corso. Se la partita è in corso serve a ricevere il tempo rimanente per quella partita, le proposte corrette, le parole rimaste ancora da raggruppare correttamente, il numero di errori ed il punteggio corrente. Se la partita è conclusa (per vittoria, sconfitta, o tempo scaduto) serve a ricevere la corretta assegnazione delle 16 parole ai 4 gruppi, il numero di proposte corrette, il numero di errori fatti, ed il punteggio ottenuto;
    - *richiedere statistiche partita*: l'utente specifica l'id univoco della partita a cui è interessato oppure che è interessato alla partita in corso. Se la partita è in corso serve a ricevere il tempo rimanente per quella partita, il numero di giocatori con quella partita ancora in corso, il numero di giocatori che hanno concluso la partita ed il numero di giocatori che hanno concluso la partita con una vittoria. Se la partita è conclusa serve a ricevere il numero di giocatori che hanno partecipato alla partita, il numero di giocatori che hanno concluso la partita, il numero di giocatori che hanno concluso la partita con una vittoria ed il punteggio medio dei giocatori che hanno partecipato alla partita.
- Operazioni di richiesta di informazioni generali (solo se logged in):
    - informazioni su classifica: in base ai parametri della richiesta può richiedere l'intera classifica di tutti gli utenti, la classifica dei top K utenti o la posizione nella classifica di un certo utente (sia esso il richiedente o meno);
    - statistiche personali: per richiedere statistiche sull'utente corrente nella seguente forma ispirata alle statistiche offerte dal New York Times.
        - Puzzles Completed: The total number of puzzles you've completed, including those you solved under 4 mistakes, those you failed, and those you did not finish.

- Win Rate %: The percentage of puzzles won by completing them in under 4 mistakes out of the total number of puzzles you've played.
- Loss Rate %: The percentage of puzzles lost by completing 4 mistakes out of the total number of puzzles you've played.
- Current Streak: The number of consecutive wins. Puzzles must be solved before the time runs out to be included in the current streak.
- Max Streak: The highest streak you've held of consecutive wins.
- Perfect Puzzles: The number of puzzles you've solved with 0 mistakes.
- Mistake Histogram: A graph showing the number of puzzles you've solved with 0 to 4 mistakes, and those you failed with 4 mistakes, and those you did not finish in time.

Quando una partita è terminata, il giocatore può decidere di disconnettersi dal server oppure continuare a giocare. Un giocatore che è logged in quando una nuova partita inizia (ad esempio perché ha partecipato alla partita precedente) si considera partecipare in automatico alla nuova partita e riceverà le nuove parole come se avesse effettuato il login durante la partita.

2.2 Il Server

La funzione principale del server del gioco *Connections* è gestire l'intera partita, coordinare i giocatori e garantire che le regole siano rispettate, fornendo anche aggiornamenti in tempo reale a tutti i partecipanti.

Il server ha a disposizione un file contenente le informazioni per ogni partita, cioè l'insieme delle parole e i gruppi di parole con significato affine, in formato JSON. Il file è fornito dai docenti ed allegato a questo progetto. Si consideri come questo file potrebbe avere grandi dimensioni in uno scenario reale e si richiede quindi di gestirlo come se le avesse anche per questo progetto (indipendentemente dalla dimensione effettiva del file fornito). Si noti anche come il file fornito contiene dati sufficienti solo per 911 partite. Lo abbiamo ritenuto un numero sufficiente per i test sul prototipo implementato (se ogni partita durasse 10 minuti, il file basterebbe per più di 6 giorni ininterrotti di test) ma tenere a mente tale limitazione nel caso improbabile di test più lunghi.

Durante il proprio periodo di attività il server crea nuove partite e le mantiene attive per un periodo di tempo prefissato. Ad ogni momento esiste un'unica partita globale attiva a cui possono partecipare tutti i giocatori logged in durante qualsiasi momento della sua durata. Mentre una partita è attiva, il server si mette in attesa di giocatori che desiderino partecipare. Quando un giocatore si connette, il server attende il log in del giocatore, per

poi inviargli tutte le informazioni sulla partita in corso (ad esempio, parole e tempo rimanente), ovviamente senza indicazioni sui raggruppamenti, lasciando ai partecipanti il compito di scoprirli. Se il giocatore aveva già partecipato alla partita in corso (ad esempio se aveva fatto logout durante la stessa partita che è ancora in corso al momento di questo log in) lo stato della partita comunicato è aggiornato di conseguenza (ad esempio il giocatore potrebbe aver già individuato dei raggruppamenti e fatto già degli errori). Un giocatore che abbia già terminato una partita ancora in corso (per vittoria o sconfitta) dovrà ovviamente aspettare l'inizio della prossima partita (che è globale ed indipendente dal singolo giocatore) prima di poter inviare proposte.

Durante la partita, il server riceve continuamente le proposte dei giocatori (sotto forma di gruppi di 4 parole). Per ogni proposta, il server valuta se le parole selezionate costituiscono un gruppo corretto e se il gruppo non è già stato individuato da quel giocatore. Il server notifica quindi il giocatore dell'esito ed aggiorna lo stato della sua partita di conseguenza (ad esempio aggiornando il numero di errori o il punteggio corrente). Si noti che l'invio di una proposta malformata, ad esempio costituita da parole già assegnate correttamente ad un gruppo o da una o più parole non parte della partita saranno notificate come errori al giocatore ma non cambieranno lo stato della partita (ad esempio non devono essere considerate come errori né influenzare il punteggio del giocatore).

La partita si conclude per un singolo giocatore quando vince o perde o per tutti i giocatori quando il tempo della partita è scaduto. In entrambi i casi il server trasmette a tutti i partecipanti la classifica di quella partita, come se un giocatore avesse richiesto l'esito di quella partita e le statistiche di quella partita (si veda sezione 2.1 per le informazioni inviate). Il server può quindi iniziare una nuova partita e continuare così fino alla propria chiusura.

È richiesta da parte del server la persistenza periodica delle informazioni sugli utenti e partite durante il proprio regolare ciclo di vita. Questi dati devono essere mantenuti consistenti ed usati per un eventuale riavvio del server dopo una precedente chiusura.

3. Specifiche per l'implementazione

Nella realizzazione del progetto devono essere utilizzate le tecnologie illustrate durante il corso. In particolare:

- gli utenti interagiscono con il servizio Connections mediante un client che utilizza un'interfaccia a linea di comando. È facoltativa e non valutata l'implementazione di un'interfaccia grafica;
- fase di registrazione:

    - (studenti nuovo ordinamento) questa fase viene implementata instaurando una connessione TCP con il server
    - (solo studenti vecchio ordinamento) questa fase viene implementata mediante Remote Method Invocation
- il client interagisce con il server, secondo il modello client-server (richieste/risposte), sulla connessione TCP persistente creata, inviando uno dei comandi elencati in sezione 2.1 secondo il formato dei messaggi JSON indicato in sezione 5. Tutte le operazioni devono avvenire su questa connessione TCP. E' richiesto che Il client gestisca questa connessione utilizzando NIO;
- il server deve essere multithreaded realizzato usando JAVA thread pooling;
- il server definisce strutture dati opportunamente sincronizzate per memorizzare le informazioni relative agli utenti e allo stato del gioco corrente o storico di giochi passati;
- quando il server deve mandare comunicazioni asincrone, ad esempio la notifica di fine partita allo scadere del tempo, le notifiche vengono inviate utilizzando *il protocollo UDP*. Il client deve quindi essere progettato in modo che sia in grado di ricevere dal server *notifiche asincrone*;
- i file per la persistenza degli utenti (potenzialmente inclusa classifica) e delle partite memorizzano le informazioni in formato JSON;
- (solo studenti vecchio ordinamento) in seguito al login il client si registra a un servizio di notifica del server per ricevere aggiornamenti tutte le volte che cambia la classifica. Il servizio di notifica deve essere implementato mediante multicast UDP, cioè il server invia un messaggio su un gruppo di multicast a cui tutti gli utenti sono registrati.

4. Modalità di svolgimento e di consegna del progetto

Il progetto deve essere eseguito individualmente e deve essere realizzato in Java. Il materiale da consegnare comprende:

- il codice dell'applicazione e di eventuali programmi utilizzati per il test delle sue funzionalità. Si tenga presente che il codice:
    - deve compilare correttamente da riga di comando (ovvero invocando direttamente il compilatore *javac*). In caso contrario, il progetto non verrà considerato valido;
    - deve essere opportunamente commentato;

        - le classi che contengono un metodo main devono contenere “Main” nel nome, es. `ServerMain.java`; per le altre classi non ci sono vincoli, ma nomi mnemonici sono ovviamente apprezzati;
        - oltre al codice sorgente, è necessario consegnare un file `JAR` eseguibile per ogni applicazione (es. un file `JAR` per i client e uno per il server);
        - i parametri di input delle applicazioni (numeri di porta, indirizzi, valori di timeout, ecc.) devono essere letti automaticamente da appositi file di configurazione da consegnare assieme al resto del codice (due file separati per client e server). Non è consentito leggere i parametri in modo "interattivo" (ovvero facendo in modo che sia il programma a chiederli dopo essere stato avviato), ne passati come parametri da linea di comando;
        - in caso di progetti realizzati con `Eclipse`, `IntelliJ IDEA` o altri `IDE`, è obbligatorio consegnare solamente il codice sorgente, rimuovendo eventuali altri file (o directory) creati dall’IDE per gestire il progetto;
        - eventuali librerie esterne utilizzate (in formato `jar`) vanno allegate al progetto;
        - i comandi che il client invia al server devono rispettare la sintassi illustrata in sezione 5;
    - una *relazione sintetica* (massimo 5 pagine) in formato pdf, deve contenere:
        - la definizione delle scelte effettuate nei punti del progetto lasciati alla personale interpretazione;
        - uno schema generale dei thread attivati sia lato server che lato client;
        - una definizione delle strutture dati utilizzate sia lato server che lato client;
        - una descrizione delle eventuali primitive di sincronizzazione utilizzate dai thread per accedere a strutture dati condivise;
        - una sezione di istruzioni su come compilare ed eseguire il progetto (librerie esterne usate, argomenti da passare al codice, sintassi dei comandi per eseguire le varie operazioni, ecc.). Questa sezione deve essere un manuale di istruzioni chiaro per gli utilizzatori del sistema.

Relazione e codice sorgente devono essere consegnati su Moodle in un unico archivio compresso in formato zip (non rar, non gz).

5. ALLEGATO 1. Specifiche delle operazioni

Il sistema utilizza un protocollo di comunicazione testuale strutturato, basato su messaggi in formato stringa. Di seguito è indicato il formato dei messaggi dal client in JSON obbligatorio (tutti gli elementi indicati devono essere presenti e con la chiave specificata), a cui possono essere aggiunti elementi opzionali se necessario/desiderato.

Operazione	Formato dati	
	Richiesta	**Risposta**

register	{
	“operation”: “register”,
	“name”: STRING,
	“psw”: STRING
	}
		Specificare correttamente codici di errore (ad esempio name già registrato) e formato JSON della risposta

updateCredentials	{
	“operation”: “updateCredentials”,
	“oldName”: STRING,
	*“newName”: STRING,
	“oldPsw”: STRING,
	*“newPsw”: STRING
	}
	*può essere utilizzato per aggiornare psw (newName non presente) o nome utente (newPsw non presente) o entrambe
		Specificare correttamente codici di errore (ad esempio oldPsw errata o newName già registrato) e formato JSON della risposta

login	{
	“operation”: “login”,
	“username”: STRING,
	“psw”: STRING
	}
		Specificare correttamente codici di errore (ad esempio psw errata) e formato JSON della risposta

logout	{
	“operation”: “logout”
	}
		Specificare correttamente codici di errore (ad esempio utente non loggato) e formato JSON della risposta

submitProposal	{
	“operation”: “submitProposal”,
	“words”: [STRING, … , STRING]
	}
		Specificare correttamente codici di errore (ad esempio proposta malformata) e formato JSON della risposta in caso che proposta sia corretta o errore

requestGameInfo	{
	“operation”: “requestGameInfo”,
	“gameId”: INT
	}
	specificare opzione per indicare partita corrente
		Specificare correttamente codici di errore (ad esempio gioco inesistente) e formato JSON della risposta con le informazioni specificate

		in sezione 2.1

requestGameStats	{
	“operation”: “requestGameStats”,
	“gameId”: INT
	}
	specificare opzione per indicare partita corrente
		Specificare correttamente codici di errore (ad esempio gioco inesistente) e formato JSON della risposta con le informazioni specificate in sezione 2.1

requestLeaderboard	{
	“operation”: “requestLeaderboard”,
	*“playerName”: STRING,
	**“topPlayers”: INT
	}
	*può essere utilizzato per ottenere ranking relativo o top K utenti
	**specificare opzione per indicare che si desiderano tutti i giocatori
		Specificare correttamente codici di errore (ad esempio giocatore inesistente) e formato JSON della risposta con le informazioni specificate in sezione 2.1

requestPlayerStats	{
	“operation”: “requestPlayerStats”
	}
		Specificare correttamente codici di errore e formato JSON della risposta con le informazioni specificate in sezione 2.1